Green Home Score & Retrofit Recommendation Engine

Explainable scoring, accuracy-first energy models and a grounded LLM explainer, for homeowners and for a lender's mortgage book

1. Problem Statement and Our Understanding

Many customers want an energy-efficient home but don't know where to start. The task is an AI system that analyses a property's characteristics, energy consumption and existing features to (1) calculate a 0-100 **Green Home Score**, (2) identify poorly performing homes, (3) recommend **feasible** retrofits (insulation, heat pumps, solar, glazing, smart controls), (4) estimate energy savings and carbon reduction, and (5) estimate the potential property-value impact. It must answer: *how green is my home, what should I improve, how much could I save, and could it raise my home's value?*

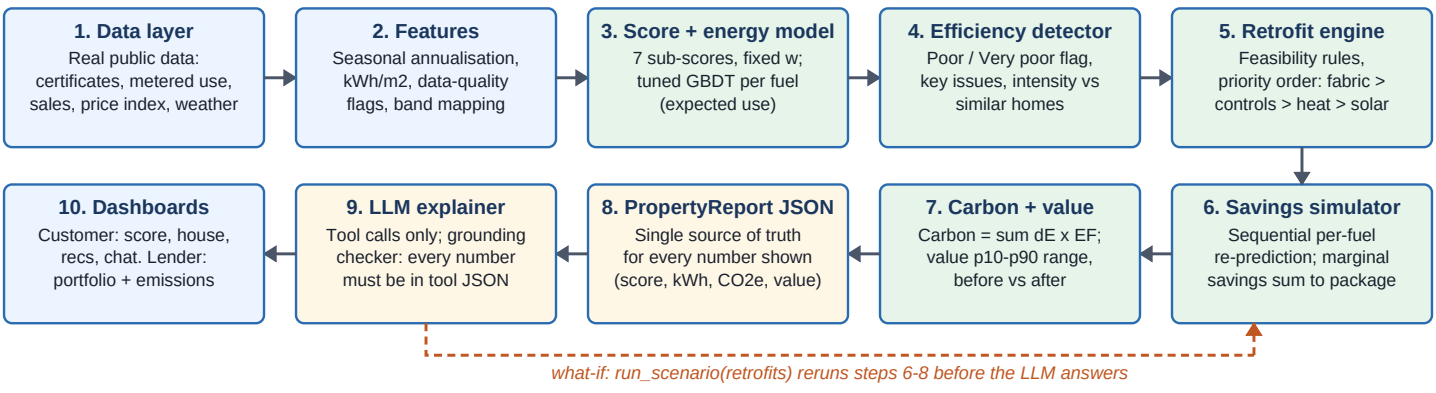
What we take from it: the requirements that make this hard

- **Numbers come from code, never the LLM.** Score, savings, carbon and value are computed by documented models; the LLM only explains them. A what-if must rerun the models first, and then the LLM explains the new outputs.
- **Per-fuel accounting.** Electricity and gas are tracked separately end to end. Purchased energy = grid import + gas; solar cuts imports, not demand; a heat pump lowers gas but raises electricity, which is reported as a *negative* saving. Carbon = sum over fuels of $dE_f \times EF_f$.
- **Interactions, not sums.** Retrofits are ordered (insulation before a heat pump), and each acts on the demand left by the previous one, so package savings are not the sum of individual estimates.
- **Fair efficiency judgement.** A large home must not be flagged just for using more energy: we judge intensity (kWh/m^2) against what *similar homes* are expected to use.
- **Honest uncertainty.** Savings are estimates; value impact is an indicative *range* for the same home on the same date, never a causal claim; missing data is reported as unavailable, never silently 0.
- **Explainable, fixed-weight score.** Seven sub-scores with fixed weights (25/20/15/15/10/10/5%), each mapping monotonic (a better state never scores lower) and documented, with each component's contribution shown.

2. Idea and Abstract

We build a **deterministic engine with a thin, grounded LLM on top**. Every property flows through one pipeline that produces a single **PropertyReport** JSON: score with per-component contributions, Poor/Very-poor flag and key issues, a prioritised list of feasible retrofits, per-fuel marginal and package savings, carbon, and an indicative value range. The core models are trained on **real public data**: a national anonymised sample of 50,000 homes with **metered** annual gas and electricity (2005-2023, including dated loft, cavity-wall and solar installs), energy performance certificates for three contrasting local authorities (a northern city, a rural off-gas-grid county and a dense inner-city borough), house-sale transactions with a regional price index, and regional monthly temperatures for heating degree days. Synthetic data is used only where none exists publicly (monthly bills per certificate home, customer IDs, smart-control level), and is **never used to report accuracy**. Because model accuracy matters most, the energy model is a tuned gradient-boosted tree per fuel with monotone constraints and calibrated p10/p90 bands. Retrofits the data cannot teach (heat pump, glazing, smart controls) use a small, sourced and clearly labelled rule layer. The LLM can only call tools (*get_property_report*, *run_scenario*, memory), and a grounding checker rejects any answer containing a number that is not in the tool output. The same reports feed a **customer dashboard** (score breakdown, heat-loss house, savings, chat) and a **lender portfolio view** (score mix, financed emissions, customers ranked by improvement potential).

3. Pipeline and Architecture



How each stage works

- **Data and features (1-2).** Monthly bills are annualised with separate seasonal weights for gas and electricity; low coverage is flagged, never filled with 0. Certificate fields are mapped to the same bands the energy model was trained on.
- **Green Home Score (3).** Seven monotonic sub-score mappings live in a versioned config, each with a source. Energy efficiency uses the recorded rating; consumption intensity uses annualised metered kWh/m^2, to avoid double-counting.
- **Energy model (3).** One model per fuel, trained on real metered consumption: a LightGBM vs CatBoost bake-off tuned with Optuna on identical cross-validation folds, monotone constraints (worse rating or less insulation never predicts lower use), and conformalised p10/p90 bands. Its residual (actual vs expected for similar homes) drives the efficiency detector (4).
- **Retrofit engine and savings (5-6).** Hard feasibility rules (e.g. no cavity fill on solid walls, no solar without roof access) give a prioritised package with High/Medium/Low suitability. The simulator applies retrofits in priority order and re-predicts each fuel after each step, so per-retrofit savings are marginal and add up to the package total.
- **Carbon and value (7).** Carbon uses official greenhouse-gas conversion factors per fuel. Value: a quantile model gives the price level, and a hedonic model's rating-band coefficients give a smooth uplift *range* for the same home and date, labelled indicative.
- **LLM and dashboards (8-10).** The LLM explains only report JSON, says 'not available' for nulls, and cites the rule that blocked an infeasible retrofit. A what-if reruns steps 6-8 first.

4. Planned Testing Methodology

Testing is layered from deterministic code to models to the LLM. Model results are only reported on held-out real data, and the test split is touched once.

Layer	What we test	How	Target / metric
Unit and invariant tests	Weights sum to 1; every sub-score mapping monotonic; sample poor home (rating E, gas boiler, single glazing) scores ~38 / Poor; missing data gives 'unavailable', not 0	pytest on score, carbon, feasibility, annualisation and schema; property-based checks over mapping grids	100% pass, run on every commit
Energy model accuracy	Annual electricity and gas prediction on homes the model never saw	Fixed 70/15/15 split of the metered sample, test set scored once; 5-fold CV for all tuning	MAE, MAPE, R^2 per fuel; must beat median-of-similar-homes and log-linear baselines
Savings validity	Predicted effect of loft, cavity-wall and solar installs	Compare with observed before/after changes in the metered sample (difference-in-differences)	Within the observed effect range; catalogue ranges as sanity bounds
Interactions and per-fuel logic	Ordered package; heat pump raises electricity; solar cuts imports only	Scenario tests on fixed homes through the full engine	Marginal savings sum to package; signs correct
Value model	Price level and rating uplift	Held-out sales; interval coverage; baseline: regional median x rating premium	p10-p90 coverage ~80%; beats baseline
Leakage audit	Every feature available at prediction time	Installs after the target year excluded; splits by customer; no post-retrofit or post-sale fields	Automated tests on the feature pipeline
LLM grounding	~50 held-out scenarios: missing or conflicting data, infeasible asks, heat-pump electricity rise	Grounding checker extracts every number and matches it to tool JSON	Groundedness >= 90%, accuracy >= 90%, unsupported claims < 5%
Fairness audit	Score, flag rate and model error by region, property type, age band and value band	Test-split breakdowns; affordability flag puts low-cost options first for high energy-burden homes	No group systematically worse; limitations documented

Reproducibility. Every result is regenerated from the repository with fixed seeds: *make data* (synthetic bills, seed 42), *make train* (energy model with baselines; cross-validation and validation only), *make test* (unit and invariant suite). The train/validation/test split is saved once and versioned, so the test set cannot drift between runs, and every constant in the configuration (score mappings, retrofit catalogue, emission factors, efficiencies) carries a *source* field.